TRADEVERSE AI

Project Proposal and Software Requirements Specification

An Intelligent Web and Mobile Stock Market Platform with AI-Based Market Analysis and Virtual Trading

Field	Details
Course	Web Programming & Mobile Applications Development
Institution / Department	Khulna University, CSE Discipline
Supervisor	Dr. Kazi Masudul Alam (Professor ,CSEKU)
Team Members	Mukesh Kumar Mandal, ID: 220242 Nuzhat Liana, ID: 220205
Prepared By	TradeVerse AI Project Team
Repository	https://github.com/Mukeshmandal815/TradeVerseAI

Contents

1. Project Overview
2. Project Objectives
3. Scope and MVP Boundary
4. User Roles and Stakeholders
5. System Boundaries
6. Requirements Specification
7. Assumptions and Constraints
8. Initial Architecture Overview
9. GitHub Repository and Contribution Guidelines
10. AI-Assisted Development Workflow
11. AI-Generated README Draft
12. Initial Task List
13. Deliverables and Acceptance Criteria
14. Development Timeline
Appendix A — Requirement Traceability Starter

1. Project Overview

TradeVerse AI is a software engineering project that gives students and early-career learners a safe, zero-risk way to practise stock trading and market analysis on Nepal Stock Exchange (NEPSE) listed companies. The MVP focuses on virtual/paper trading: users receive simulated funds, browse real or mocked NEPSE market data, place buy/sell orders, and track a full portfolio, while a dedicated AI service layers short-term trend prediction and financial-news sentiment analysis on top of the market data.

The project is designed for a university software project context: it should be demonstrable locally, use a clear and testable scope, and provide evidence of requirements engineering, architecture planning, implementation discipline, and team collaboration.

Item	Baseline
Problem	Students and early-career learners lack an accessible, modern, education-focused way to practise NEPSE trading; existing tools are either full brokerage platforms unsuitable for beginners or plain spreadsheets with no live data, charting, or intelligence layer.
Proposed solution	A web and mobile platform that connects to NEPSE market data, lets users trade with simulated funds, tracks a full virtual portfolio, and layers AI-based trend prediction and news-sentiment analysis on top.
Primary users	Student project team, course supervisor/evaluator, and future learners who want to practise NEPSE trading without financial risk.
MVP target	NEPSE-listed companies only; virtual/paper trading, not real brokerage execution.
Local development posture	No real-money integration, paid brokerage subscription, or production deployment is required for the MVP; a mock market-data provider is acceptable when live NEPSE data is unavailable.

2. Project Objectives

The project begins with an initiation phase whose goal is not to build the full product immediately, but to agree on the product boundary, establish collaboration practices, and create a requirement set that can drive MVP implementation. The overall objectives are to:

- Define the project problem statement, objectives, MVP boundary, stakeholders, and user roles.
- Draft a structured SRS with functional requirements, non-functional requirements, constraints, assumptions, and acceptance criteria.
- Set up the GitHub repository with README, contribution guidelines, issue labels, branch rules, and an initial task list.
- Define the AI-assisted development workflow the team will use responsibly throughout the project.
- Create a first architecture overview covering the web app, mobile app, backend, database, market-data integration, AI service, and Docker-based local runtime.
- Prepare deliverables for supervisor/team review and turn the approved requirements into prioritized GitHub issues.

3. Scope and MVP Boundary

The MVP intentionally narrows the technical problem so the team can produce a working, defensible project within a university timeline. The core value is safe virtual trading with AI-assisted market insight, not a full brokerage or multi-exchange platform.

In Scope for MVP	Out of Scope for MVP
NEPSE-listed companies and their market data.	Other stock exchanges or multi-market support.
Virtual/paper trading with simulated funds only.	Real brokerage integration or real-money transactions.
Web app (Next.js) and mobile app (Flutter) on a shared NestJS backend.	Enterprise SSO, organization-wide policy enforcement, or marketplace distribution.
PostgreSQL with Prisma ORM for trading, portfolio, and market data.	Production-scale multi-tenant billing or high-availability infrastructure.
AI trend prediction (XGBoost, LSTM) and FinBERT-based news sentiment.	Guaranteed-accuracy financial advice or autonomous trading decisions.
Admin dashboard for user, market-data, and model-performance monitoring.	Options/derivatives trading and user-defined strategy backtesting.

MVP Outcomes

- A registered user can open a TradeVerse AI account and receive a virtual starting balance automatically.
- The backend can source NEPSE market data (live or mocked) and expose company listings, prices, and history.
- A user can place market/limit buy and sell orders that are validated and executed entirely on the backend.
- The system tracks holdings, cash balance, and realized/unrealized profit-loss in a live portfolio view.
- The AI service can produce an explainable trend prediction and sentiment reading for a selected stock.

4. User Roles and Stakeholders

Role	Responsibilities	Key Needs
Registered User (Trader)	Browses market data, manages a watchlist, places virtual trades, and reviews portfolio performance and AI insights.	Accurate market data, a trustworthy trading engine, and clearly explained AI output.
Administrator	Manages users, monitors market-data ingestion, and reviews AI model performance and audit logs.	Reliable admin tooling, system visibility, and safe access control.
Project Team Member	Plans, builds, tests, documents, and reviews the MVP.	Clear tasks, reproducible setup, shared coding standards, and requirement traceability.
Course Supervisor / Evaluator	Reviews SRS quality, scope feasibility, technical approach, and final demonstration.	Professional documentation, measurable acceptance criteria, and evidence of engineering process.
Market Data Source Maintainer	Provides or approximates NEPSE price and historical data consumed by the platform.	A stable provider interface, documented rate limits, and graceful fallback to mock data.

5. System Boundaries

TradeVerse AI sits between NEPSE market data and a learner practising trading decisions. It does not replace a real brokerage, an official exchange system, or a licensed financial advisor. The system provides simulation, analysis evidence, and AI-assisted explanation, while final trading judgment and real-world financial decisions stay with the user.

Boundary Area	Inside TradeVerse AI	Outside TradeVerse AI
Identity and Access	Store user accounts, roles, and session/JWT tokens for the platform.	Real brokerage account creation, bank identity verification, or external KYC providers.
Market Data	Consume NEPSE price/history through a MarketDataProvider abstraction (live or mock).	Direct exchange-level order routing or official NEPSE settlement systems.
Trading and Portfolio	Simulate order placement, execution, holdings, and profit/loss with virtual funds only.	Real-money deposits, withdrawals, or brokerage-grade trade execution.
AI Assistance	Use XGBoost/LSTM and FinBERT locally or via the AI microservice to draft trend signals and sentiment.	Guaranteed financial advice, automated real-money trading, or regulatory-grade investment recommendations.
Persistence	Use PostgreSQL/Redis for MVP data such as users, orders, trades, holdings, and predictions.	Production data warehousing, billing records, or organization-scale audit retention.

6. Requirements Specification

Requirement IDs should be used in GitHub issues, commits, pull requests, and acceptance reviews. Priorities use P0 for must-have MVP baseline, P1 for important but deferrable, and P2 for useful enhancements.

6.1 Functional Requirements — Core Platform

Requirement	Acceptance Indicator
The system shall provide secure user registration, login, logout, and JWT-based session management.	A user can register, verify email, log in, and receive a valid access/refresh token pair.
The system shall support role-based access control distinguishing USER and ADMIN accounts.	Admin-only routes reject requests carrying a USER-role token with a 403 response.
The system shall allow users to browse NEPSE-listed companies with search and sector filters.	A user can search by ticker or company name and filter results by sector.
The system shall display current and historical price data for a selected stock, including OHLC, volume, and turnover.	The stock detail page renders current price fields and a historical chart with selectable ranges (1D–5Y).
The system shall support a pluggable MarketDataProvider abstraction with a mock provider for offline development.	Switching the configured provider does not require changes to consuming modules.
The system shall issue each new user a fixed virtual starting balance.	A newly registered user's portfolio shows a cash balance of NPR 1,000,000.

6.2 Functional Requirements — Trading and Portfolio

Requirement	Acceptance Indicator
The system shall allow users to place MARKET and LIMIT buy/sell orders.	An order submitted through the UI is persisted with the correct type, side, quantity, and status.
The system shall validate and execute all trading calculations exclusively on the backend.	Manipulated client-side price or quantity values are rejected or recalculated server-side.
The system shall maintain a Holding record per user per stock with average price and quantity.	Buying the same stock twice updates a single Holding row with a recalculated average price.
The system shall compute and display unrealized and realized profit/loss per holding and in aggregate.	The portfolio view shows P/L figures consistent with current price and average cost.
The system shall record every executed trade in a queryable transaction history.	A user can view a chronological list of past trades with price, quantity, and timestamp.
The system shall support watchlists and configurable price alerts with real-time notification delivery.	A triggered alert produces a notification visible in the UI within a reasonable delay.

6.3 Functional Requirements — AI and Analytics

Requirement	Acceptance Indicator
The AI service shall generate short-term trend predictions using XGBoost and LSTM models.	The prediction endpoint returns a symbol, predicted price, horizon, prediction score, and trend signal.
The AI service shall evaluate models using MAE, RMSE, MAPE, and directional accuracy on a chronological hold-out set.	Evaluation metrics are stored against the model version in the Model table.

The system shall ingest financial news and classify sentiment as positive, negative, or neutral using FinBERT.	A processed news item has an associated NewsSentiment record with a sentiment label and score.
The AI dashboard shall present bullish/bearish/neutral groupings, prediction scores, and news sentiment.	The dashboard renders without errors for a stock with existing predictions and sentiment data.
The system shall clearly label all AI output as experimental/educational and not financial advice.	Every AI-generated prediction or sentiment badge in the UI displays a visible disclaimer.

6.4 Non-Functional Requirements

Category	Requirement	Target / Measure
Usability	The MVP interface shall be understandable to a learner reviewing market data and trades.	Core actions (search, trade, view portfolio) reachable within 3 clicks from the dashboard.
Security	Passwords, JWT secrets, and market-data API keys shall not be committed to the repository.	.env.example is committed; real .env files are ignored.
Privacy	A user's trading data shall be isolated from other users.	A user cannot query another user's portfolio or orders via the API.
Performance	The MVP shall render the dashboard and stock detail pages quickly for a demo dataset.	Target: under 2 seconds for a sample dataset in local demo.
Reliability	The backend shall return readable errors for invalid orders, missing data, or a market-data provider outage.	Failure states appear in the API response and UI status.
Maintainability	The codebase shall separate web, mobile, backend, AI service, and shared-type concerns.	Monorepo structure and contribution rules reflect this separation.
Portability	The system shall be runnable on a development machine using documented Node.js, Docker, PostgreSQL, Redis, and Python setup.	README contains setup steps and an environment-variable guide.
Explainability	Predictions shall include a rationale rather than only a number.	Each prediction includes a prediction score, trend signal, and horizon.

7. Assumptions and Constraints

7.1 Assumptions

- A usable NEPSE data source (official, licensed, or reasonably approximated) is available, or a mock provider is used during development.
- Users interact with the platform to learn trading concepts, not to execute real financial transactions.
- The development team can run Node.js, Python, Docker, PostgreSQL, and Redis locally.
- AI predictions are treated as educational signals reviewed by the team before being presented as trend signals, not investment advice.
- Team members have working familiarity with TypeScript, Python, and relational databases from prior coursework.

7.2 Constraints

- The MVP will not process real money, deposits, or withdrawals under any circumstance.
- No paid brokerage subscription, data-vendor license, or production cloud hosting is assumed for the coursework MVP baseline.
- NEPSE data licensing and rate limits may restrict the frequency and volume of live data retrieval during testing.
- Model training data volume is limited to what is available for NEPSE-listed companies, which may constrain prediction accuracy.
- Trading logic must remain backend-authoritative; client-submitted values are never trusted for price or balance calculations.

8. Initial Architecture Overview

The initial architecture separates user interaction, backend orchestration, market-data integration, AI analysis, persistence, and cross-cutting concerns. The design favors local development, testability, and a clean swap point for a future real-brokerage integration.

Layer / Component	Technology	Responsibility
Frontend Web App	Next.js, TypeScript, Tailwind CSS, shadcn/ui	Dashboard, stock search/detail, trade ticket, portfolio, watchlist, and AI dashboard screens.
Mobile App	Flutter, Dart	Same feature set as the web app, consuming identical backend APIs.
Backend API	NestJS, Node.js, TypeScript	REST/WebSocket endpoints, auth, trading engine, portfolio calculation, and persistence orchestration.
Database	PostgreSQL, Prisma ORM	Stores users, stocks, orders, trades, holdings, watchlists, alerts, notifications, predictions, and audit logs.
Cache / Realtime	Redis	Caches market data, sessions/rate limiting, and pub-sub for live price ticks.

Market Data Integration	MarketDataProvider abstraction (NepseProvider / MockMarketDataProvider)	Fetches and normalizes live or mock NEPSE price and historical data.
AI Service	Python, FastAPI, XGBoost, LSTM (TensorFlow/Keras), FinBERT	Trend prediction, news sentiment classification, and model evaluation.
Runtime / Tooling	Docker, Docker Compose, GitHub Actions	Repeatable local setup, CI checks, and a future deployment pipeline.

8.1 High-Level Data Flow

1. User logs in and lands on the dashboard, which loads market summary and portfolio data.
2. User searches or browses to a stock detail page; the backend requests current/historical data via the MarketDataProvider.
3. User submits a buy/sell order; the backend validates funds and quantity and executes the trade at the authoritative price.
4. Backend updates Holding, PortfolioSnapshot, and Trade records, then pushes a WebSocket order-update event.
5. The AI service periodically, or on demand, computes predictions and sentiment, which the backend caches and exposes via /ai endpoints.
6. Web and mobile clients render updated portfolio value, profit/loss, and AI dashboard insights.
7. Triggered price alerts generate notifications delivered via WebSocket, push, or email.

8.2 Proposed Repository Structure

```
tradeverse-ai/
├── apps/
│   ├── web/                # Next.js + TypeScript + Tailwind frontend
│   └── mobile/             # Flutter mobile app
├── services/
│   ├── api/                # NestJS backend API
│   └── ai-service/         # FastAPI AI/ML microservice
├── packages/
│   ├── types/              # Shared DTOs, types, validation schemas
│   ├── config/             # Shared eslint/tsconfig/env schema
│   └── ui/                 # Shared shadcn/ui component wrappers
├── prisma/
│   ├── schema.prisma
│   └── migrations/
├── docs/
│   ├── srs/                # SRS, diagrams, review notes
│   └── architecture/       # Architecture decisions and ADRs
├── docker/
│   └── docker-compose.yml  # Local development services
├── scripts/                # Seed scripts, mock-data generators
├── tests/                  # Cross-cutting e2e tests
├── .github/
│   ├── ISSUE_TEMPLATE/
│   └── pull_request_template.md
├── README.md
├── CONTRIBUTING.md
```

```
└── .env.example
```

9. GitHub Repository and Contribution Guidelines

The repository should demonstrate professional collaboration. Every requirement implemented during the project should be traceable to an issue, branch, pull request, and review outcome.

9.1 Repository Setup

- Create/maintain the GitHub repository at the linked URL (tradeverse-ai).
- Initialize README.md, CONTRIBUTING.md, .gitignore, .env.example, and a license file if required by the course.
- Add issue labels: priority:P0, priority:P1, priority:P2, type:requirement, type:frontend, type:backend, type:ai, type:mobile, type:docs, status:blocked, status:review.
- Create issue templates for feature, bug, documentation, and research-spike tasks.
- Create a pull request template requiring a linked issue, screenshots where relevant, tests/checks, and documentation updates.
- Keep database credentials, JWT secrets, and any market-data API keys out of version control.

9.2 Branching and Commit Rules

Rule	Guideline
Main branch	main should contain reviewed, stable project work only.
Feature branches	Use feature/<short-topic>, fix/<short-topic>, docs/<short-topic>, or spike/<short-topic>.
Commits	Use concise conventional-style messages such as feat(api): add order endpoint or docs(srs): update requirements.
Pull requests	Every PR must link to an issue and include testing notes or a reason testing is not applicable.
Reviews	At least one teammate should review changes affecting requirements, architecture, or shared code.
Secrets	Never commit .env, database passwords, JWT secrets, or market-data API keys.

9.3 Pull Request Checklist

- Linked issue or requirement ID is included.
- Scope matches the issue and avoids unrelated refactoring.
- Local setup or test command has been run where applicable.
- Documentation or README changes are included when behavior/setup changes.
- UI changes include a screenshot or short description of the visible result.
- Secrets and generated local data are not included.

10. AI-Assisted Development Workflow

AI may speed up requirement drafting, boilerplate generation, and documentation, but it does not replace stakeholder review. The team uses AI to structure, clarify, and refine work while keeping human ownership of scope and trading-logic decisions.

Step	Activity	Human Review Gate
1	Collect raw inputs: project idea, supervisor feedback, team notes, and constraints.	Confirm the notes are accurate and complete.
2	Ask AI to convert notes into draft goals, scope, roles, and candidate requirements.	Remove unsupported assumptions and mark uncertain items.
3	Rewrite each requirement using shall-style wording, priority, and acceptance indicator.	Check that every requirement is testable and feasible.
4	Map each requirement to architecture components and initial tasks.	Verify traceability and avoid over-scoping.
5	Create GitHub issues from approved requirements.	Assign owner, label priority, and link to the SRS section.
6	Use AI to summarize changes after review meetings.	Team lead approves final wording before updating the baseline.

10.1 AI Use Rules

- AI-generated content must be reviewed and edited by the project team before submission.
- Do not paste database credentials, access tokens, personal data, or private repository secrets into AI prompts.
- Use AI outputs as drafts; keep trading-logic decisions, tradeoffs, and final acceptance criteria owned by humans.
- Record important requirement changes in the revision history and GitHub issues.
- When AI explains a prediction or sentiment result, label the explanation as advisory and keep the underlying data visible.

11. AI-Generated README Draft

The following README content can be adapted for the repository. It is intentionally concise and should evolve as the implementation becomes real.

README Outline

- Project title and one-paragraph description.
- MVP features and explicit limitations.
- Technology stack.
- Architecture overview.
- Local setup prerequisites.
- Environment variables and market-data provider configuration.
- Development commands.
- Testing and AI evaluation notes.
- Contribution workflow.
- Project status and university context.

Starter README Content

```
# TradeVerse AI

TradeVerse AI is a virtual/paper-trading platform for Nepal Stock Exchange
(NEPSE) listed companies. Users trade with simulated funds while an AI
microservice explains short-term trend signals and financial-news sentiment.

TradeVerse AI never processes real money and never presents AI output as
guaranteed financial advice.

## Problem
Learners rarely get a safe, realistic way to practise stock trading. Real
brokerages carry financial risk, and spreadsheets offer no live data, charts,
or intelligence layer.

## Solution
TradeVerse AI combines a modular trading engine with a bounded AI layer:
- A MarketDataProvider abstraction sources live or mock NEPSE data.
- A backend-authoritative trading engine executes virtual buy/sell orders.
- XGBoost and LSTM models generate short-term trend predictions.
- FinBERT scores financial-news sentiment per company.
- Every AI output is labelled experimental / educational in the UI.

## Features
- Email/password authentication with JWT sessions and password reset flow.
- Dashboard, market, stock detail, trade, portfolio, watchlist, orders,
  transactions, news, AI, profile, settings, and admin screens.
- NEPSE company list, search, sector filters, OHLC, and historical charts.
- Virtual wallet, MARKET/LIMIT orders, holdings, and realized/unrealized P/L.
- Watchlists, price alerts, and real-time notifications.
- AI dashboard: bullish/bearish/neutral view, prediction score, trend signal,
  and news sentiment.
```

````
- Admin dashboard for users, audit logs, and model-performance monitoring.

## Architecture
```text
Next.js / Flutter UI
        |
   NestJS API
        |
PostgreSQL + Redis
        |
MarketDataProvider + XGBoost/LSTM + FinBERT (FastAPI AI service)
```
The backend owns trading logic, balance validation, and persistence. The AI
service only explains data and drafts trend signals — it never executes trades.

## Tech Stack
- Frontend: Next.js, TypeScript, Tailwind CSS, shadcn/ui, ECharts, TanStack
  Query, React Hook Form, Zod.
- Backend: NestJS, TypeScript, Prisma, PostgreSQL, Redis, class-validator.
- AI: Python, FastAPI, Pandas, NumPy, Scikit-learn, XGBoost, TensorFlow/Keras,
  FinBERT.
- Mobile: Flutter, Dart.
- DevOps: Docker, Docker Compose, GitHub Actions, Nginx.

## Installation
Requirements:
- Node.js 20 or newer
- Python 3.11 or newer
- Docker Desktop
- PostgreSQL 15 or compatible (or use the provided Docker service)

Set up:
```bash
pnpm install
cp .env.example .env
pnpm prisma:migrate
pnpm prisma:seed
```
Update `.env` with your local database URL, a long `JWT_SECRET`, and your
chosen MARKET_DATA_PROVIDER (`mock` or `nepse`).

## Running Locally
```bash
docker compose up -d      # postgres, redis
pnpm dev                  # web + api
uvicorn app.main:app --reload   # ai-service
```
- Web: `http://localhost:3000`
- API: `http://localhost:4000/api/v1`
- AI service: `http://localhost:8000`

## Useful Commands
```bash
pnpm dev
pnpm build
pnpm lint
````
````

```
pnpm test
pnpm prisma:migrate
pnpm prisma:seed
pytest services/ai-service
```

Documentation
Start with: `docs/SRS.md`, `docs/ARCHITECTURE.md`, `docs/DATABASE.md`,
`docs/API.md`, `docs/AI_MODEL.md`, `docs/DEPLOYMENT.md`, `docs/TESTING.md`.

Limitations
TradeVerse AI supports NEPSE-listed companies and virtual trading only.
Real brokerage execution, multi-exchange support, and options/derivatives
simulation are documented as future work.

License
Educational use — course project for Khulna University, CSE Discipline.

12. Initial Task List

The following initial tasks should be entered into GitHub issues after supervisor/team review. Owners are placeholders until the team assigns responsibilities.

Task	Output
Finalize project title, problem statement, and MVP boundary.	Approved overview section.
Review and update SRS requirements with supervisor feedback.	SRS v1.0 baseline.
Create GitHub repository, README, CONTRIBUTING, .gitignore, and .env.example.	Initialized repository.
Create issue labels, issue templates, and pull request template.	Collaboration workflow in GitHub.
Create frontend/backend/mobile project skeletons.	Next.js, NestJS, and Flutter apps run locally.
Design the Prisma schema for users, stocks, orders, trades, holdings, and predictions.	Initial schema/ER notes.
Research a NEPSE data source and build a MockMarketDataProvider spike.	Provider spike notes and sample output.
Implement the backend-authoritative order execution and P/L calculation logic.	Trading-engine spike and unit tests.
Test XGBoost/LSTM prediction and FinBERT sentiment locally with sample data.	Prompt/model notes and sample response.
Define a sample stock and news fixture for demo purposes.	Sample market data + news fixture.
Plan future CI checks and deployment path.	Architecture note / ADR.

13. Deliverables and Acceptance Criteria

ID	Deliverable	Acceptance Criteria
D-01	SRS Draft	Supervisor/team can review the document and each requirement is testable, prioritized, and traceable.
D-02	GitHub Repository	Team members can clone the repo, understand setup, and create work items consistently.
D-03	Initial Architecture Note	Architecture explains data flow and technology responsibilities without over-scoping the MVP.
D-04	AI Workflow	AI use is documented as advisory, reviewed by humans, and safe for secrets/privacy.
D-05	Initial Backlog	Every P0 task maps to a project objective or MVP requirement.

13.1 Acceptance Checklist

- Project overview, objectives, and MVP scope are approved by the team.
- Functional and non-functional requirements are clear, prioritized, and measurable.
- System boundaries explicitly limit the MVP to NEPSE-listed companies and virtual trading.
- GitHub repository is created with README, contribution rules, and issue tracking structure.

- Market-data provider approach is documented without requiring a paid data-vendor subscription.
- Architecture overview identifies all major stack components and expected data flow.
- Initial task list is entered into GitHub issues and assigned to team members.
- Supervisor/team feedback is recorded and reflected in the SRS revision history.

14. Development Timeline

The project is planned over 18 weeks across 10 development phases, ending with a fully documented, demo-ready system:

Phase	Weeks	Deliverable
1	1	Requirements & design: SRS, ER diagram, architecture, wireframes, GitHub setup.
2	1.5	Core system: authentication, user management, navigation, dashboard shell, database.
3	2	Market & trading: NEPSE data provider, stock pages, charts, virtual wallet, buy/sell — MVP checkpoint.
4	2.5	Watchlist, alerts, notifications, portfolio analytics, transaction history.
5	3	AI: data preprocessing, XGBoost, LSTM, evaluation, prediction API, news sentiment, AI dashboard.
6	3.5	Mobile application (Flutter MVP).
7	6	Admin dashboard.
8	7	Testing and security hardening.
9	8	Docker, CI/CD, deployment.
10	9-10	Final documentation, evaluation reports, user manual, presentation, demo.

Appendix A. Requirement Traceability Starter

Primary Component	Demo Evidence
Backend API / Trading Engine	Order placement, execution, and balance validation proof on a sample account.
MarketDataProvider	Live and mock data retrieval examples for a sample NEPSE stock.
AI Prediction Service	Sample prediction output with score, horizon, and trend signal for a test stock.
Sentiment Pipeline	Sample news item processed into a sentiment label and score.
Frontend Web/Mobile App	Dashboard and stock-detail screens showing live data, trade flow, and AI dashboard.
Repository / Configuration	.env.example present; real .env ignored; CI workflow file committed.